4. A multi-specialty hospital wants to assist its junior doctors with a rule-based diagnostic support tool that identifies the likelihood of a disease based on patient-reported symptoms and test results. Using historical patient records as the training dataset, select and build a suitable decision-based learning model that captures diagnostic rules from the data and apply it to determine the condition of a newly admitted patient.

Decision Tree is a supervised machine learning classification technique that uses a tree-like structure of decision rules to classify data into different categories. In this experiment, it is used to predict whether a patient's condition is Malignant or Benign based on medical test measurements.

The Program:-

- Uses the Breast Cancer Wisconsin dataset available in Scikit-learn.
- Selects important medical features for disease classification.
- Builds a Decision Tree Classifier using the training data.
- Evaluates the model using Accuracy, Precision, Recall, and F1-score.
- Displays a Confusion Matrix and the learned Decision Tree.
- Applies the trained model to an unseen patient record to predict the condition and prediction probability.

Program

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import ConfusionMatrixDisplay

# Load dataset
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Find Top 8 features
tree = DecisionTreeClassifier(max_depth=4, random_state=42)
tree.fit(X_train, y_train)

importance = pd.Series(
    tree.feature_importances_, index=X.columns
```

```
).sort_values(ascending=False)

features = importance.head(8).index.tolist()

print("\nTop 8 Features:")
for i, feature in enumerate(features, 1):
    print(i, ".", feature)

# Select Top 8 features
X_train = X_train[features]
X_test = X_test[features]

# Train final Decision Tree
model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Performance
print("\nPerformance Metrics:")
print("Accuracy :", round(accuracy_score(y_test, y_pred), 3))
print("Precision:", round(precision_score(y_test, y_pred), 3))
print("Recall   :", round(recall_score(y_test, y_pred), 3))
print("F1 Score :", round(f1_score(y_test, y_pred), 3))

# Confusion Matrix
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred,
    display_labels=["Malignant", "Benign"]
)
plt.title("Confusion Matrix")
plt.show()

# Decision Tree
plt.figure(figsize=(16, 8))
plot_tree(
    model,
    feature_names=features,
    class_names=["Malignant", "Benign"],
    filled=True
)
plt.show()

# New Patient
patient = X_test.iloc[[0]]
prediction = model.predict(patient)[0]
probability = model.predict_proba(patient).max()

print("\nNew Patient Diagnosis:")
```

```
print("Condition:", "Benign" if prediction == 1 else "Malignant")
print("Prediction Probability:", round(probability, 2))
```